



REPUBLIQUE TUNISIENNE
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITE DE SFAX
FACULTE DES SCIENCES DE SFAX
Departement d'Informatique

Licence en Ingénierie des Systemes Informatiques
Specialite : Ingénierie des Réseaux et Systemes

Conception et Développement d'une Plateforme de Cyber Threat Intelligence (CTI) avec Surveillance Automatisée du Dark Web

Realise par :

Amal Karaoud
Mahdi Ghozzi

Encadrement :

M. Yassine Masmoudi
Encadrant professionnel

Mme Emna Charfi Ep Hammami
Encadrante academique

Table des matières

I	Contexte Stratégique, État de l'Art et Architecture Fondamentale	1
I.1	Problématique : L'Asymétrie Tactique et la Crise Cognitive	1
I.1.a	L'obsolescence du modèle réactif	1
I.1.b	L'infobésité des centres opérationnels de sécurité	2
I.2	État de l'Art : Évaluation Critique de l'Écosystème CTI	2
I.2.a	Solutions institutionnelles et à code ouvert	2
I.2.b	Solutions commerciales propriétaires	3
I.2.c	Synthèse et positionnement	4
I.3	Ingénierie des Exigences	4
I.3.a	Exigences fonctionnelles	4
I.3.b	Exigences non fonctionnelles	5
I.4	Vue d'Ensemble Architecturale : Approche Macro-Modulaire	5
I.4.a	Pilier 1 — Matrice d'Acquisition	6
I.4.b	Pilier 2 — Moteur Cognitif et Orchestration	7
I.4.c	Pilier 3 — Matrice de Persistance Polyglotte	8
I.4.d	Pilier 4 — Restitution et Interopérabilité	9
II	Justification Stratégique et Technologique de l'Architecture	12
II.1	La collecte des données	12
II.1.a	Le langage principal de développement : Python	12
II.1.b	Les outils de collecte : Scrapy, Playwright et BeautifulSoup4	13
II.1.c	Le format d'échange en cybersécurité : STIX 2.1 et TAXII	14
II.1.d	Les sources publiques de référence	14
II.1.e	L'exportation des alertes : MISP, OpenCTI et Splunk	15
II.2	Le moteur d'analyse et les services applicatifs	16
II.2.a	Le serveur d'application : FastAPI et Uvicorn	16
II.2.b	L'interface de requête : GraphQL avec Strawberry	16
II.2.c	L'analyse automatique des textes : SciBERT et NER	17
II.2.d	L'assistance conversationnelle : Google GenAI et Deep Chat React	17
II.2.e	La génération de règles de détection : YARA et Sigma	18
II.2.f	La mise à jour en temps réel : WebSockets	18
II.2.g	L'exécution asynchrone des tâches : Celery	19
II.3	Les bases de données et les services de stockage	20
II.3.a	La base relationnelle principale : PostgreSQL	20

II.3.b	Le stockage documentaire : MongoDB	20
II.3.c	La base orientée graphe : Neo4j	21
II.3.d	Le moteur de recherche textuelle : Elasticsearch	21
II.3.e	La base vectorielle : Qdrant	22
II.3.f	Le cache mémoire : Redis	22
II.4	L'interface utilisateur et les visualisations	23
II.4.a	Le cadre de développement de l'interface : Next.js et React	23
II.4.b	Le style, l'accessibilité et l'état local : Tailwind CSS, Radix UI et Zustand	24
II.4.c	Les animations d'interface : Framer Motion	24
II.4.d	La visualisation relationnelle en trois dimensions : Cytoscape.js, Three.js et Force-Graph	25
II.4.e	La cartographie géospatiale : Mapbox, Deck.gl et GeoIP	25
II.4.f	Les graphiques statistiques : Recharts et Chart.js	26
II.4.g	La génération de rapports PDF : Puppeteer et Express	26
II.5	La sécurité, la documentation et l'exploitation	27
II.5.a	Le contrôle d'accès et la validation des entrées : JWT, RBAC et Zod	27
II.5.b	La documentation interactive : OpenAPI et Swagger	27
II.5.c	Le démarrage et l'orchestration des services : scripts Batch et Docker Com- pose	28

Chapitre I

Contexte Stratégique, État de l'Art et Architecture Fondamentale

I.1 Problématique : L'Asymétrie Tactique et la Crise Cognitive

L'évolution actuelle du paysage des cybermenaces impose de revoir en profondeur les modèles défensifs traditionnels. Nous constatons une asymétrie tactique structurelle entre les capacités offensives des groupes adverses, qui agissent avec agilité, coordination et un niveau croissant de sophistication, et les dispositifs défensifs des organisations, souvent limités par des architectures réactives et fragmentées. Cette asymétrie ne correspond pas à un simple déséquilibre quantitatif de ressources : elle révèle un écart cognitif fondamental dans la capacité à anticiper, contextualiser et neutraliser les menaces avant leur concrétisation opérationnelle.

I.1.a L'obsolescence du modèle réactif

Le modèle défensif historique repose sur une logique de détection *a posteriori* : l'organisation subit une intrusion, identifie l'incident, puis déploie des contre-mesures correctives. Ce cycle réactif présente une latence incompatible avec la vitesse des attaques modernes. Le *Dwell Time*, c'est-à-dire l'intervalle temporel entre la compromission initiale d'un système et sa détection effective par l'équipe de défense, constitue l'indicateur le plus révélateur de cette inadéquation. Les rapports sectoriels aboutissent à un constat alarmant : ce délai se compte encore en dizaines de jours, voire en mois, dans de nombreuses organisations. Pendant cette période d'exposition, l'adversaire consolide ses accès, exfiltre des données stratégiques, élève ses privilèges et prépare les phases finales de son opération, qu'il s'agisse du déploiement d'un rançongiciel, d'un sabotage industriel ou d'un espionnage persistant.

Nous identifions trois facteurs aggravants qui renforcent cette obsolescence :

- **La prolifération des surfaces d'attaque** : l'adoption massive des infrastructures en nuage, la généralisation du travail à distance et l'interconnexion des systèmes industriels dissolvent le périmètre défensif traditionnel. Chaque point de connexion devient un vecteur potentiel d'intrusion que les architectures périmétriques classiques couvrent difficilement.
- **La professionnalisation des groupes adverses** : les acteurs de la menace opèrent désormais selon des modèles économiques structurés, comme le rançongiciel en tant que

service, le courtage d'accès initiaux et les places de marché d'exploits, qui abaissent fortement le seuil d'entrée technique tout en augmentant le volume et la sophistication des attaques.

- **La fragmentation des signaux d'alerte** : les indicateurs précurseurs d'une attaque sont dispersés dans une multitude de sources hétérogènes, comme les journaux système, les flux de renseignement, les forums clandestins et les réseaux sociaux, ce qui rend leur corrélation manuelle impraticable à grande échelle.

I.1.b L'infobésité des centres opérationnels de sécurité

Les Centres Opérationnels de Sécurité (SOC) actuels font face à une crise cognitive d'une ampleur inédite. Le volume de données de sécurité produit chaque jour par les capteurs, les sondes et les systèmes de journalisation dépasse de plusieurs ordres de grandeur la capacité d'analyse humaine. Cette infobésité crée un paradoxe : une très grande quantité d'informations brutes coexiste avec une pénurie de renseignement réellement actionnable.

Les analystes SOC consacrent une part disproportionnée de leur temps à des tâches de tri, de déduplication et de qualification élémentaire, au lieu de se concentrer sur l'analyse stratégique et la chasse proactive aux menaces. Le taux de faux positifs, souvent supérieur à 90% dans les environnements non optimisés, entraîne une fatigue décisionnelle qui réduit la vigilance et allonge mécaniquement les temps de réponse.

L'aspect le plus critique de cette problématique réside dans l'exploitation des signaux faibles issus des couches profondes de l'Internet, comme les forums clandestins, les places de marché illicites et les canaux de communication chiffrés. Ces espaces constituent l'environnement opérationnel dans lequel les adversaires planifient leurs actions, échangent des outils offensifs, négocient des accès compromis et publient des données exfiltrées. L'absence de mécanismes automatisés de collecte, de normalisation et de corrélation de ces signaux prive les défenseurs d'un avantage d'anticipation déterminant.

Nous formulons ainsi la problématique centrale de ce travail : **comment concevoir une plateforme souveraine de renseignement sur les cybermenaces capable de réduire l'asymétrie tactique en transformant le flux brut de données hétérogènes en renseignement contextualisé, corrélé et immédiatement exploitable par les équipes de défense ?**

I.2 État de l'Art : Évaluation Critique de l'Écosystème CTI

L'écosystème du renseignement sur les cybermenaces s'organise autour de deux familles de solutions, que nous évaluons selon des critères d'exhaustivité fonctionnelle, de capacité cognitive, de souveraineté et d'interopérabilité.

I.2.a Solutions institutionnelles et à code ouvert

Plusieurs plateformes à code ouvert se sont imposées comme des références dans le partage et la gestion du renseignement sur les menaces. Nous examinons ici les deux initiatives les plus structurantes.

MISP (*Malware Information Sharing Platform*) constitue la plateforme de référence pour le partage communautaire d'indicateurs de compromission (IoC). Lancée et maintenue par le

CERT gouvernemental luxembourgeois, elle bénéficie d’une adoption institutionnelle large et d’un écosystème particulièrement riche de taxonomies et de galaxies d’attributs. Toutefois, notre évaluation met en évidence plusieurs limites structurelles :

- **Orientation centrée sur l’indicateur atomique** : MISP est très performant pour gérer des IoC discrets, comme les empreintes cryptographiques, les adresses réseau et les noms de domaine, mais il ne propose pas nativement de couche de corrélation sémantique capable de reconstruire des chaînes d’attaque complexes et des relations latentes entre entités.
- **Absence de moteur cognitif intégré** : la plateforme ne dispose pas de capacités d’analyse automatisée par intelligence artificielle. L’enrichissement contextuel et la priorisation des menaces reposent entièrement sur l’expertise humaine de l’analyste.
- **Interface de restitution fonctionnelle mais austère** : la couche de visualisation se limite à des tableaux et à des graphes de corrélation élémentaires, sans proposer de cartographie géospatiale, de projection en trois dimensions des réseaux criminels ou de génération narrative automatisée.

OpenCTI, développée sous l’impulsion de l’Agence nationale de la sécurité des systèmes d’information (ANSSI) et du CERT-EU, adopte une approche plus structurée en s’appuyant nativement sur le modèle de données STIX 2.1. Cette orientation donne à la plateforme une meilleure capacité de modélisation relationnelle, puisqu’elle permet de représenter les campagnes, les groupes adverses, les tactiques et les vulnérabilités comme des entités interconnectées dans un graphe de connaissances. Toutefois, nous relevons les limites suivantes :

- **Complexité de déploiement et d’opération** : l’architecture distribuée d’OpenCTI exige une infrastructure importante et des compétences avancées d’administration, ce qui la rend difficilement accessible aux organisations de taille intermédiaire.
- **Capacités cognitives embryonnaires** : même si des connecteurs d’enrichissement existent, la plateforme ne possède pas de moteur d’intelligence artificielle intégré capable de produire des analyses narratives, de détecter des tendances émergentes ou de noter dynamiquement la criticité des menaces.
- **Collecte limitée des sources profondes** : OpenCTI n’intègre pas nativement d’agents de collecte automatisés capables d’opérer dans les couches profondes et clandestines de l’Internet. Cette fonction est confiée à des connecteurs tiers dont la couverture reste partielle.

1.2.b Solutions commerciales propriétaires

Le segment commercial du renseignement sur les cybermenaces est dominé par des éditeurs qui proposent des plateformes intégrées à forte valeur analytique, comme Recorded Future, Mandiant Advantage et CrowdStrike Falcon Intelligence. Ces solutions offrent effectivement des capacités avancées : collecte multi-sources à grande échelle, enrichissement automatisé, corrélation par apprentissage automatique et interfaces de restitution sophistiquées. Toutefois, leur adoption soulève des préoccupations fondamentales :

- **Opacité algorithmique** : les moteurs d’analyse, les modèles de notation et les algorithmes de corrélation fonctionnent comme des composants opaques. L’organisation utilisatrice ne peut ni auditer leur logique interne ni évaluer leurs biais éventuels. Cette opacité est incompatible avec les exigences de traçabilité et de reproductibilité scienti-

fique.

- **Dépendance et perte de souveraineté** : le recours à ces plateformes crée une dépendance stratégique vis-à-vis d'un éditeur étranger, à la fois pour la continuité opérationnelle et pour la confidentialité des données de renseignement traitées. Les données de menace propres à l'organisation, qui révèlent indirectement ses vulnérabilités et ses priorités défensives, transitent par des infrastructures tierces échappant à tout contrôle souverain.
- **Rigidité fonctionnelle et coût prohibitif** : les modèles de licence commerciaux imposent des coûts récurrents élevés et une souplesse limitée pour personnaliser les flux d'analyse, les taxonomies et les formats de sortie.

1.2.c Synthèse et positionnement

Notre analyse de l'état de l'art met en évidence un espace d'opportunité architecturale clairement défini. Les solutions à code ouvert offrent la transparence et la souveraineté, mais présentent un déficit cognitif structurel. Les solutions commerciales compensent ce déficit analytique au prix de la souveraineté et de l'auditabilité. Aucune solution existante ne réalise la convergence entre souveraineté totale, capacité cognitive avancée, collecte proactive des couches profondes et restitution visuelle immersive. Ce constat fonde la légitimité de notre démarche de conception.

I.3 Ingénierie des Exigences

Le passage de l'analyse critique de l'existant à la conception de notre solution exige une formalisation rigoureuse des exigences fonctionnelles et non fonctionnelles. Nous organisons ces exigences en deux catégories complémentaires, chacune résultant directement des lacunes identifiées dans l'état de l'art.

1.3.a Exigences fonctionnelles

Nous définissons les capacités opérationnelles que la plateforme doit impérativement assurer :

- **EF-1 — Acquisition multi-sources hétérogène** : la plateforme doit intégrer des agents de collecte automatisés capables d'ingérer simultanément des flux structurés provenant de sources institutionnelles publiques, comme les bases de vulnérabilités, les flux de renseignement normalisés et les avis de sécurité gouvernementaux, ainsi que des données non structurées extraites des couches profondes de l'Internet, comme les forums clandestins, les places de marché illicites et les canaux de communication chiffrés.
- **EF-2 — Normalisation et structuration sémantique** : toute donnée ingérée doit passer par une chaîne de normalisation qui la transforme en objets de renseignement conformes aux standards d'interopérabilité reconnus, afin de garantir l'homogénéité du corpus analytique, quelle que soit la source d'origine.
- **EF-3 — Analyse cognitive automatisée** : un moteur d'intelligence artificielle intégré doit assurer la catégorisation thématique automatique, l'extraction d'entités nommées, la détection de tendances émergentes et la génération de synthèses narratives exploitables, afin de réduire la charge cognitive des analystes.
- **EF-4 — Notation dynamique de la criticité** : la plateforme doit calculer en temps réel un score de criticité pour chaque menace identifiée, en intégrant des paramètres

contextuels, comme le secteur d'activité ciblé, la géolocalisation, la proximité temporelle et la fiabilité de la source, et ce score doit pouvoir être ajusté selon le profil de risque de l'organisation utilisatrice.

- **EF-5 — Visualisation immersive et multi-dimensionnelle** : la couche de restitution doit offrir des capacités de cartographie géospatiale mondiale, de représentation en graphes de réseaux criminels et relationnels, de projection chronologique et de génération automatisée de rapports exportables.
- **EF-6 — Interopérabilité bidirectionnelle** : la plateforme doit permettre l'exportation native du renseignement produit vers les systèmes de gestion des événements de sécurité (SIEM) et les plateformes tierces de partage d'IoC, ainsi que la réception de flux entrants en provenance de ces mêmes écosystèmes.

I.3.b Exigences non fonctionnelles

Nous précisons les attributs de qualité architecturale que la plateforme doit garantir :

- **ENF-1 — Souveraineté et maîtrise du déploiement** : l'ensemble de la chaîne de traitement, depuis l'acquisition jusqu'à la restitution, doit pouvoir être déployé et exploité au sein d'une infrastructure souveraine contrôlée par l'organisation, sans dépendance à des services tiers opaques.
- **ENF-2 — Modularité et couplage faible** : l'architecture doit adopter un découpage modulaire strict dans lequel chaque composant fonctionnel communique par des interfaces contractualisées, afin de permettre le remplacement, la mise à l'échelle ou l'évolution indépendante de chaque module sans propagation d'effets de bord.
- **ENF-3 — Extensibilité horizontale** : chaque couche fonctionnelle doit pouvoir monter en charge par ajout d'instances parallèles, sans modification architecturale structurelle.
- **ENF-4 — Sécurité intrinsèque** : la plateforme doit intégrer nativement des mécanismes de contrôle d'accès à base de rôles, de limitation du débit d'interrogation, d'authentification par jetons cryptographiques et de journalisation exhaustive de toute interaction.
- **ENF-5 — Performance et réactivité** : les temps de réponse aux interrogations analytiques et les latences d'ingestion doivent rester compatibles avec un usage opérationnel en temps réel, y compris sous charge soutenue.
- **ENF-6 — Auditabilité et traçabilité** : chaque décision algorithmique, chaque transformation de donnée et chaque score produit par le moteur cognitif doit être traçable et reproductible, afin de garantir la confiance des analystes dans les résultats produits.

La conjonction de ces exigences, en particulier l'impératif de souveraineté (ENF-1), la nécessité d'un moteur cognitif intégré (EF-3) et l'obligation de collecte proactive des couches profondes (EF-1), rend impossible l'adoption d'une solution existante sans compromis rédhibitoire. Nous justifions ainsi la nécessité de concevoir une architecture originale, que nous présentons dans la section suivante.

I.4 Vue d'Ensemble Architecturale : Approche Macro-Modulaire

Notre plateforme repose sur une architecture systémique à couplage faible, organisée en modules fonctionnels interconnectés par des flux de données asynchrones. Afin de rendre cette

complexité plus lisible, nous présentons cette architecture selon quatre piliers fonctionnels qui traduisent le parcours logique de la donnée, depuis son acquisition brute jusqu'à sa restitution sous forme de renseignement actionnable.

I.4.a Pilier 1 — Matrice d'Acquisition

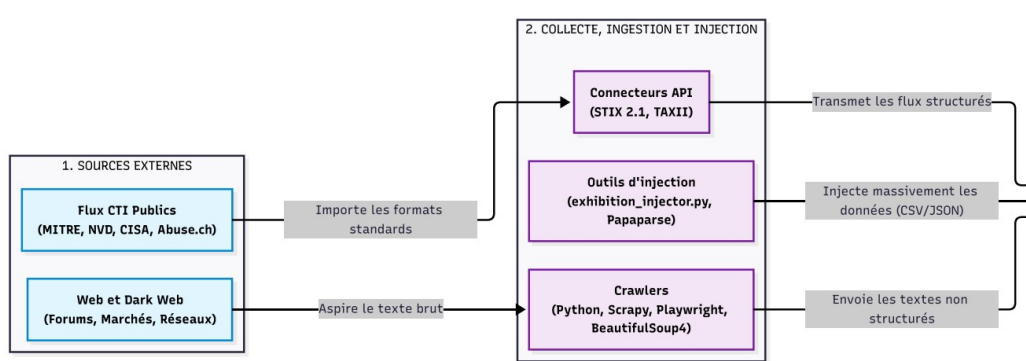


FIGURE I.1 – Pilier 1 : Flux d'ingestion des sources ouvertes et profondes.

La matrice d'acquisition constitue le premier niveau fonctionnel de la plateforme. Elle assure l'ingestion systématique de données de menace issues de deux familles de sources complémentaires.

Le premier vecteur d'acquisition agit sur les **sources ouvertes et institutionnelles**. Des connecteurs normalisés interrogent périodiquement les bases publiques de vulnérabilités, les flux gouvernementaux de renseignement, les registres communautaires d'indicateurs de compromission et les avis de sécurité publiés par les autorités compétentes. Ces flux, déjà partiellement structurés, alimentent directement la chaîne de normalisation.

Le second vecteur cible les **sources profondes et clandestines**. Des agents de collecte automatisés, conçus pour agir dans les couches profondes de l'Internet, extraient des données brutes depuis les forums clandestins, les places de marché illicites et les canaux de communication utilisés par les groupes adverses. Ces agents intègrent des mécanismes d'isolation et de compartimentation qui garantissent la sécurité opérationnelle du processus de collecte.

Un troisième composant, non visible sur cet extrait restreint, assure la **collecte locale et les essais contrôlés**. Nous avons conçu un environnement d'injection et d'expérimentation qui permet l'exécution supervisée de scénarios offensifs dans des environnements cloisonnés, afin d'alimenter la base de connaissances avec des données comportementales empiriques. Des profils d'injection paramétrables définissent les conditions d'exécution, comme l'isolation, la redirection et le leurre, tandis que des connecteurs normalisés assurent la conformité des données produites avec les standards d'échange en vigueur.

L'ensemble de ces vecteurs d'acquisition produit un flux continu de données hétérogènes, illustré par les flux sortants à droite du schéma, qui converge vers un bus de normalisation avant d'être transmis vers la matrice d'orchestration.

I.4.b Pilier 2 — Moteur Cognitif et Orchestration

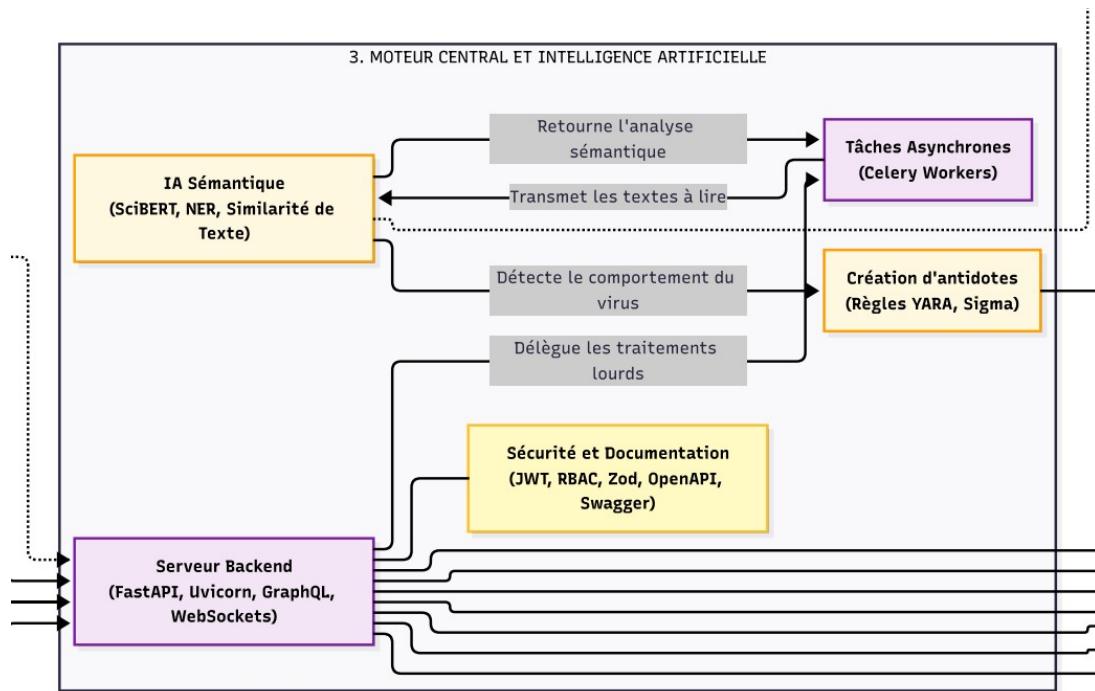


FIGURE I.2 – Pilier 2 : Architecture du moteur de traitement sémantique et d’orchestration.

En recevant les flux bruts issus de la couche d’acquisition, représentés par les flèches entrantes à gauche, le moteur cognitif constitue le noyau analytique de la plateforme. Nous avons structuré ce pilier autour de trois fonctions complémentaires qui transforment le flux de données normalisées en renseignement contextualisé.

La **fonction d’analyse thématique** exploite un modèle de langage de grande envergure, spécialisé par ajustement fin sur un corpus de cybersécurité. Ce modèle assure la catégorisation automatique des menaces par domaine thématique, l’extraction des entités nommées, comme les groupes adverses, les vulnérabilités, les vecteurs d’attaque et les cibles sectorielles, ainsi que la génération de synthèses narratives structurées. Nous avons conçu cette fonction afin qu’elle produise un renseignement directement exploitable par l’analyste, sans nécessiter de retraitement manuel.

La **fonction de notation algorithmique** calcule en continu un score de criticité multicritères pour chaque objet de renseignement. Ce score intègre la fiabilité estimée de la source, la fraîcheur temporelle de l’indicateur, la couverture géographique de la menace, le secteur d’activité ciblé et la récurrence observée. Un mécanisme de surveillance active détecte les variations brusques de ces paramètres et déclenche des alertes proactives.

La **fonction de composition et de formalisation** assure la mise en forme du renseignement produit selon les formats d’échange normalisés. Elle génère des objets de renseignement structurés, représentés par les flux sortants à droite et en bas, prêts à être persistés ou exportés vers des systèmes tiers.

L’orchestration de ces fonctions s’appuie sur une couche de services dorsaux qui expose des interfaces programmatiques contractualisées. Cette couche intègre nativement les mécanismes de

sécurité transversaux : contrôle d'accès à base de rôles, authentification par jetons, limitation du débit d'interrogation, documentation normalisée des interfaces et communication bidirectionnelle en temps réel.

I.4.c Pilier 3 — Matrice de Persistance Polyglotte

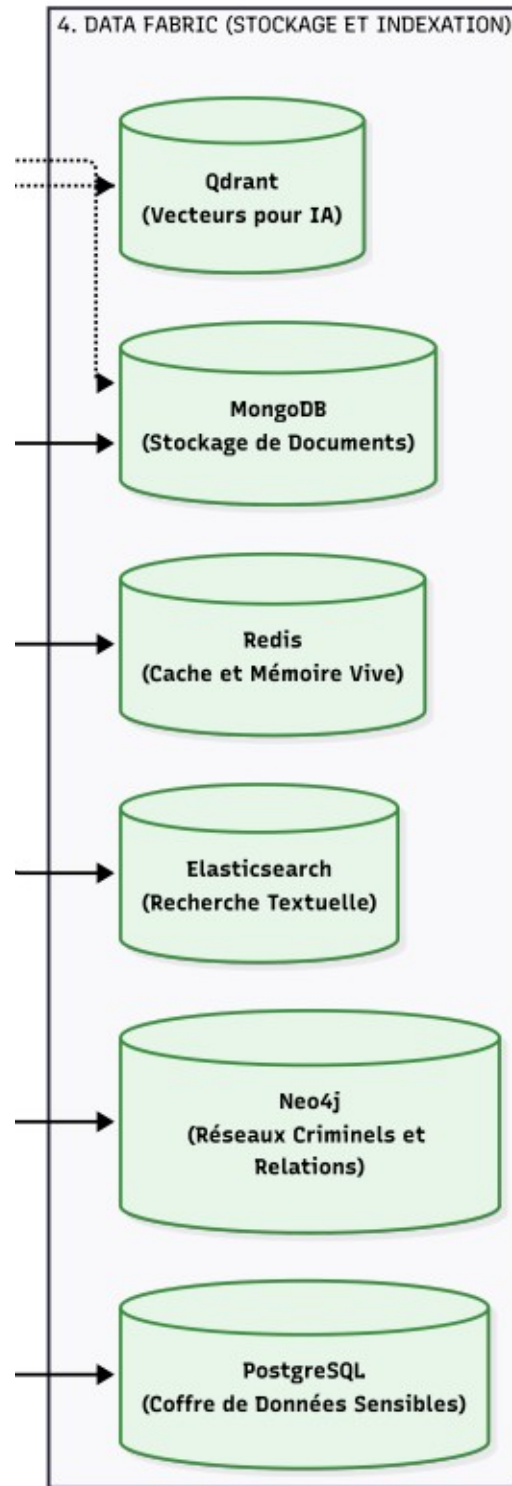


FIGURE I.3 – Pilier 3 : Matrice de persistance et stockage spécialisé.

Alimentée par les flux sortants du moteur cognitif, représentés par les flèches entrantes à gauche de la figure, nous avons conçu la couche de persistance selon un paradigme polyglotte dans lequel chaque famille de données est confiée au modèle de stockage le plus adapté à sa nature et à ses schémas d'interrogation. Cette stratégie optimise les performances d'accès tout en préservant la cohérence transactionnelle globale.

La matrice de persistance se décompose en quatre compartiments fonctionnels :

- **Stockage en mémoire volatile** : un compartiment de cache haute performance assure la persistance temporaire des données fréquemment sollicitées, comme les sessions actives, les résultats de requêtes récurrentes et l'état des files d'attente, ce qui réduit la latence des opérations critiques.
- **Indexation textuelle intégrale** : un moteur de recherche distribué indexe l'ensemble du corpus de renseignement sous forme de documents semi-structurés, en offrant des capacités d'interrogation textuelle avancée avec pondération par pertinence, recherche approximative et agrégation analytique.
- **Persistance relationnelle et graphe** : un compartiment de stockage orienté graphe modélise les relations complexes entre entités de renseignement, comme les liens entre groupes adverses, campagnes, infrastructures d'attaque et vulnérabilités exploitées, et autorise des requêtes de traversée ainsi que la découverte de chemins d'attaque. Un compartiment relationnel transactionnel assure, en parallèle, la persistance des données structurées à forte exigence d'intégrité, comme les référentiels utilisateurs, les configurations et les historiques d'audit.
- **Stockage vectoriel sémantique** : un compartiment spécialisé conserve les représentations vectorielles des documents de renseignement, ce qui permet des recherches par similarité sémantique exploitées par le moteur cognitif pour la corrélation contextuelle et la détection d'analogies entre menaces.

I.4.d Pilier 4 — Restitution et Interopérabilité

Le quatrième pilier assure la transformation du renseignement persisté et orchestré par le moteur central en représentations visuelles exploitables, ainsi que sa diffusion vers les écosystèmes de sécurité tiers. Il se divise en deux blocs fonctionnels distincts.

I.4.d.1 Interface Utilisateur et Visualisation

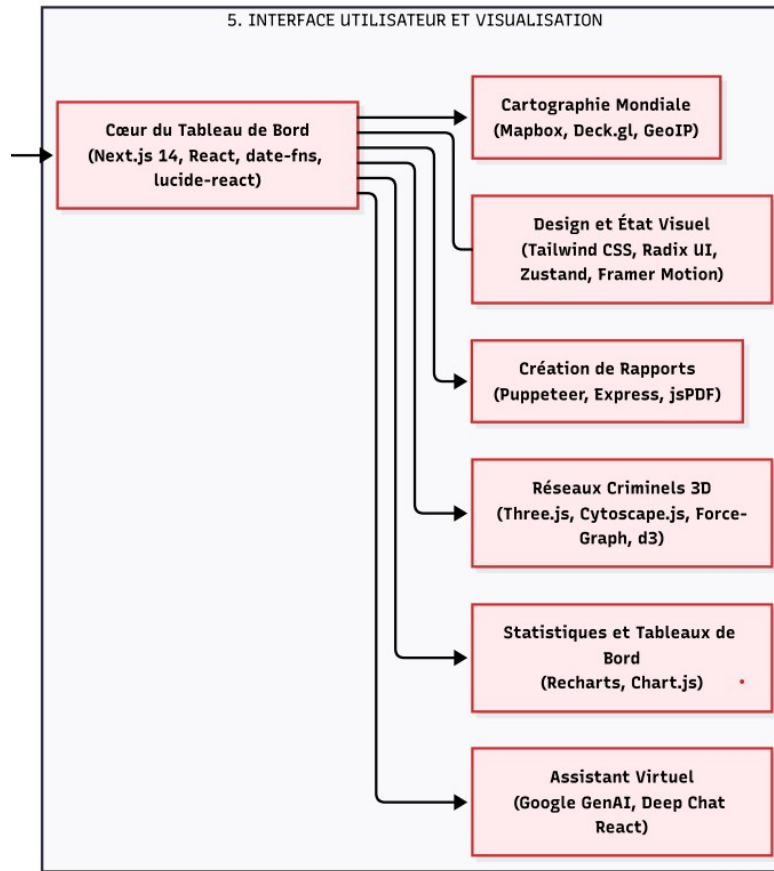


FIGURE I.4 – Pilier 4 (Restitution) : Fonctions de visualisation et tableau de bord.

La couche de restitution visuelle constitue le tableau de bord central de la plateforme. Nous avons structuré cette couche autour de six fonctions de visualisation complémentaires :

- Une **cartographie géospatiale mondiale** projette la distribution géographique des menaces détectées sur un planisphère interactif, ce qui permet une appréciation immédiate de la couverture territoriale des campagnes adverses.
- Un **moteur de conception d'état visuel** restitue en temps réel les indicateurs clés de performance du renseignement : volume d'ingestion, répartition thématique et évolution temporelle des scores de criticité.
- Un **générateur de rapports** produit automatiquement des documents de synthèse exportables, formatés selon les standards de communication institutionnelle.
- Un **module de projection de réseaux en trois dimensions** permet la visualisation immersive des graphes relationnels entre entités de renseignement, ce qui facilite la compréhension des structures organisationnelles adverses et des chaînes d'attaque complexes.
- Des **tableaux statistiques dynamiques** offrent des capacités d'agrégation, de filtrage et de comparaison temporelle sur l'ensemble du corpus de renseignement.
- Un **narrateur visuel** restitue les analyses produites par le moteur cognitif sous forme de récits structurés associant texte et représentations graphiques.

I.4.d.2 Exportation vers Partenaires

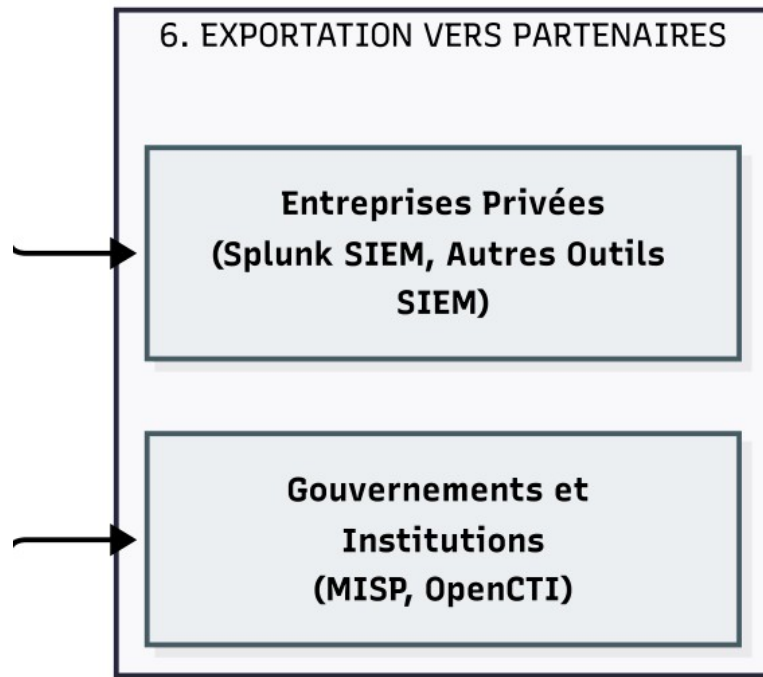


FIGURE I.5 – Pilier 4 (Interopérabilité) : Exportation du renseignement qualifié.

La couche d'interopérabilité assure la diffusion du renseignement produit vers deux catégories de consommateurs. D'une part, des connecteurs dédiés permettent l'exportation automatisée vers les systèmes de gestion des événements de sécurité (SIEM) déployés dans les infrastructures des organisations utilisatrices, ce qui assure l'intégration opérationnelle du renseignement dans les processus de détection et de réponse existants. D'autre part, des mécanismes d'abonnement et de notification alimentent les plateformes communautaires de partage d'indicateurs, comme MISP et OpenCTI, contribuant ainsi à l'enrichissement de l'écosystème collaboratif de renseignement.

Notre architecture réalise ainsi la convergence, identifiée comme absente dans l'état de l'art, entre souveraineté de déploiement, capacité cognitive intégrée, collecte proactive des sources profondes et restitution visuelle immersive. Chaque pilier fonctionnel opère de manière autonome grâce au couplage faible inter-modulaire, tout en participant à une chaîne de valeur cohérente qui transforme le signal brut en renseignement stratégique actionnable. Nous détaillons, dans le chapitre suivant, les choix technologiques et les mécanismes d'implémentation qui concrétisent cette architecture fonctionnelle.

Chapitre II

Justification Stratégique et Technologique de l'Architecture

Dans ce chapitre, nous présentons les choix technologiques retenus pour construire la plateforme de cyber threat intelligence. Pour chaque choix, nous suivons la même démarche : nous exposons le besoin fonctionnel, nous décrivons le rôle de la solution, nous la comparons à une autre option, puis nous justifions le choix final. L'objectif est de présenter une architecture cohérente, lisible et adaptée aux contraintes du projet.

II.1 La collecte des données

II.1.a Le langage principal de développement : Python

Notre besoin : Nous avons besoin d'un langage capable de traiter de grands volumes de texte, d'automatiser la collecte de données et d'intégrer des fonctions d'intelligence artificielle.

Explication simple : **Python** est un langage largement utilisé pour l'analyse de données, le traitement du langage et l'apprentissage automatique. Il offre une syntaxe lisible et un grand nombre de bibliothèques spécialisées, ce qui facilite le développement d'une chaîne de traitement complète.

Solution comparée	Avantages	Limites	Statut
JavaScript / Node.js	Adapté au développement web et à l'exécution d'interfaces réactives.	Moins riche en bibliothèques spécialisées pour le traitement avancé des données et des modèles d'IA.	Écarté
Python	Écosystème mature pour l'analyse de données, l'automatisation et l'intelligence artificielle.	Demande une bonne organisation du code pour maintenir de bonnes performances.	Retenu

TABLE II.1 – Comparaison des langages de développement principaux

Justification : Python a été retenu car il permet de développer dans un même environnement la collecte, le traitement des contenus et les modules d'analyse. Ce choix réduit la complexité de l'architecture et améliore la cohérence de l'ensemble.

II.1.b Les outils de collecte : *Scrapy*, *Playwright* et *BeautifulSoup4*

Notre besoin : Nous devons pouvoir collecter automatiquement des contenus publiés sur le Web visible et sur le Dark Web, y compris lorsque certaines pages utilisent des mécanismes de blocage ou de rendu dynamique.

Explication simple : **Scrapy** organise la collecte à grande échelle. **Playwright** pilote un navigateur afin d'accéder aux pages qui chargent leur contenu de manière interactive. **BeautifulSoup4** extrait ensuite les informations utiles à partir du code de la page.

Solution comparée	Avantages	Limites	Statut
Scripts simples de collecte	Mise en place rapide et coût réduit.	Peu robustes face aux sites dynamiques et aux mécanismes de protection contre l'automatisation.	Écarté
Scrapy + Playwright + BeautifulSoup4	Combine rapidité, compatibilité avec les pages complexes et extraction structurée des contenus.	Mise en œuvre plus exigeante et consommation plus élevée de ressources.	Retenu

TABLE II.2 – Comparaison des solutions de collecte de données

Justification : Cette combinaison offre un bon équilibre entre volume de collecte, robustesse technique et qualité d'extraction. Elle répond donc aux contraintes réelles d'une collecte sur des

sources hétérogènes et parfois restrictives.

II.1.c Le format d'échange en cybersécurité : STIX 2.1 et TAXII

Notre besoin : Nous devons produire des données compréhensibles par d'autres outils de sécurité et pouvant être partagées sans retraitement important.

Explication simple : **STIX 2.1** est un standard de description des menaces. Il permet de représenter les acteurs, les indicateurs, les campagnes et les relations entre ces éléments de manière uniforme. **TAXII** est le protocole qui facilite l'échange de ces données entre systèmes.

Solution comparée	Avantages	Limites	Statut
Format propriétaire interne	Peut être ajusté précisément aux besoins du projet.	Faible interopérabilité avec les outils et partenaires externes.	Écarté
Standard STIX/TAXII	Facilite l'échange, la compréhension et la réutilisation des informations de menace.	Implique une modélisation rigoureuse et le respect de contraintes strictes.	Retenu

TABLE II.3 – Comparaison des formats de structuration des menaces

Justification : Le recours à STIX/TAXII garantit l'interopérabilité de la plateforme. Il permet de diffuser les résultats vers des partenaires ou des outils tiers sans dépendre d'un format propre au projet.

II.1.d Les sources publiques de référence

Notre besoin : Avant d'identifier de nouvelles menaces, nous devons intégrer des sources reconnues afin de disposer d'une base fiable de comparaison et de validation.

Explication simple : La plateforme interroge des sources publiques telles que **CISA**, **MITRE** et **Abuse.ch**. Ces référentiels fournissent des données déjà qualifiées sur les vulnérabilités, les techniques d'attaque et les indicateurs malveillants.

Solution comparée	Avantages	Limites	Statut
Une seule base privée payante	Intégration simplifiée et source unique à administrer.	Dépendance forte à un fournisseur unique et couverture potentiellement limitée.	Écarté
Croisement de plusieurs sources publiques	Meilleure couverture, recoupement des informations et réduction du risque d'erreur.	Augmente le volume de données à normaliser et à trier.	Retenu

TABLE II.4 – Comparaison des stratégies d'alimentation en renseignement

Justification : Le croisement de plusieurs sources améliore la fiabilité des résultats et élargit la vision de la menace. Ce choix est donc mieux adapté à une plateforme d'analyse qui doit limiter les angles morts.

II.1.e L'exportation des alertes : MISP, OpenCTI et Splunk

Notre besoin : Nous devons pouvoir transmettre rapidement les résultats de la plateforme vers les outils déjà utilisés par les équipes de défense.

Explication simple : MISP et OpenCTI facilitent le partage de renseignement sur les menaces entre organisations. Splunk permet d'exploiter les alertes dans des dispositifs de supervision et de détection déjà en place.

Solution comparée	Avantages	Limites	Statut
Exportation manuelle	Mise en œuvre simple et sans développement supplémentaire.	Trop lente et peu fiable pour un usage opérationnel continu.	Écarté
Exportation automatisée (MISP/Splunk)	Permet une réutilisation rapide des résultats dans les outils métiers.	Nécessite des interfaces d'échange formalisées et sécurisées.	Retenu

TABLE II.5 – Comparaison des méthodes de diffusion des alertes

Justification : L'utilité opérationnelle d'une plateforme CTI dépend de sa capacité à alimenter directement les dispositifs de défense. L'automatisation de l'export réduit les délais et améliore l'intégration dans les processus existants.

II.2 Le moteur d'analyse et les services applicatifs

II.2.a Le serveur d'application : *FastAPI* et *Uvicorn*

Notre besoin : Nous devons disposer d'un serveur central capable de recevoir de nombreuses requêtes et d'y répondre avec un temps d'attente limité.

Explication simple : **FastAPI** est un cadre de développement Python adapté aux interfaces de programmation modernes. **Uvicorn** exécute ce serveur et gère efficacement plusieurs connexions en parallèle.

Solution comparée	Avantages	Limites	Statut
Django ou Flask	Solutions bien connues et simples à prendre en main.	Moins adaptées, dans ce contexte, à des échanges intensifs et fortement concurrents.	Écarté
FastAPI avec Uvicorn	Bon niveau de performance, typage clair et gestion adaptée des traitements concurrents.	Mise en œuvre plus exigeante sur le plan technique.	Retenu

TABLE II.6 – Comparaison des serveurs d'application

Justification : Ce choix répond au besoin de réactivité de la plateforme tout en restant cohérent avec l'écosystème Python utilisé dans le reste du projet.

II.2.b L'interface de requête : *GraphQL* avec *Strawberry*

Notre besoin : Nous devons permettre à l'interface cliente de demander uniquement les données nécessaires afin de limiter les transferts inutiles et d'améliorer la réactivité des tableaux de bord.

Explication simple : **GraphQL** permet au client de sélectionner précisément les champs attendus dans la réponse. **Strawberry** fournit une implémentation de GraphQL bien intégrée à Python.

Solution comparée	Avantages	Limites	Statut
REST API classique	Architecture simple et largement diffusée.	Peut conduire à envoyer plus de données que nécessaire pour une vue donnée.	Écarté
GraphQL avec Strawberry	Réponses mieux adaptées aux besoins de l'interface et réduction du volume de données échangées.	Conception plus complexe qu'une interface REST simple.	Retenu

TABLE II.7 – Comparaison des modes d'échange entre client et serveur

Justification : GraphQL améliore la précision des échanges et limite les transferts superflus. Ce point est important pour une interface riche, composée de vues nombreuses et évolutives.

II.2.c L'analyse automatique des textes : SciBERT et NER

Notre besoin : Nous devons identifier automatiquement, dans des contenus textuels, les éléments utiles à l'analyse tels que les acteurs, les organisations, les outils ou les cibles.

Explication simple : SciBERT est un modèle de traitement du langage fondé sur des représentations contextuelles. La technique de NER (reconnaissance d'entités nommées) permet d'extraire automatiquement les termes importants présents dans un texte.

Solution comparée	Avantages	Limites	Statut
Lecture manuelle	Bonne maîtrise qualitative sur un faible volume.	Impossible à maintenir à grande échelle sur des flux continus.	Écarté
SciBERT et NER	Extraction automatisée, plus rapide et exploitable sur de grands ensembles de documents.	Nécessite des données d'entraînement et des réglages adaptés au domaine.	Retenu

TABLE II.8 – Comparaison des méthodes d'analyse des textes

Justification : Cette approche permet d'automatiser un travail d'analyse volumineux tout en conservant un bon niveau de précision. Elle est donc adaptée à un contexte de surveillance continue.

II.2.d L'assistance conversationnelle : Google GenAI et Deep Chat React

Notre besoin : Nous voulions que les analystes puissent interroger la plateforme en langage naturel afin d'obtenir rapidement des synthèses ou des réponses ciblées.

Explication simple : **Google GenAI** fournit les capacités de génération et d'interprétation du langage. **Deep Chat React** offre le composant d'interface utilisé pour afficher l'échange entre l'utilisateur et le système.

Solution comparée	Avantages	Limites	Statut
Recherche textuelle simple	Mise en œuvre rapide.	Peu adaptée aux questions complexes ou formulées librement.	Écarté
Google GenAI et Deep Chat React	Facilite les échanges en langage naturel et améliore l'accès à l'information.	Repose sur un service externe et sur une gestion rigoureuse des accès.	Retenu

TABLE II.9 – Comparaison des modes d'assistance à l'utilisateur

Justification : Une interface conversationnelle réduit le temps d'accès à l'information pour des utilisateurs qui ne maîtrisent pas nécessairement la structure interne des données.

II.2.e La génération de règles de détection : YARA et Sigma

Notre besoin : Nous devons convertir les résultats de l'analyse en règles directement exploitables par des outils de détection et de supervision.

Explication simple : **YARA** sert à décrire des signatures applicables aux fichiers ou aux contenus. **Sigma** sert à formaliser des règles de détection applicables à des journaux et à des événements de sécurité.

Solution comparée	Avantages	Limites	Statut
Alerte textuelle uniquement	Facile à lire pour un analyste humain.	Ne peut pas être appliquée directement par un outil de sécurité.	Écarté
Génération YARA / Sigma	Permet un passage plus rapide de l'analyse à l'action défensive.	Demande une traduction technique rigoureuse des résultats produits.	Retenu

TABLE II.10 – Comparaison des méthodes de production des règles de détection

Justification : La valeur opérationnelle du système augmente lorsque les résultats peuvent être intégrés directement dans les moyens de détection existants.

II.2.f La mise à jour en temps réel : WebSockets

Notre besoin : Nous devons permettre aux analystes de recevoir les nouvelles alertes sans recharger manuellement l'interface.

Explication simple : Les **WebSockets** maintiennent une connexion ouverte entre le navigateur et le serveur. Cette connexion permet d’envoyer immédiatement une information nouvelle dès qu’elle est disponible.

Solution comparée	Avantages	Limites	Statut
Interrogation périodique (polling)	Implémentation simple.	Produit des délais d’affichage et des requêtes répétitives peu efficaces.	Écarté
WebSockets	Mise à jour rapide de l’interface et meilleur confort d’usage.	Gestion plus fine nécessaire pour maintenir des connexions stables.	Retenu

TABLE II.11 – Comparaison des mécanismes de mise à jour de l’interface

Justification : Dans un contexte de cybersécurité, la rapidité de diffusion d’une alerte a une incidence directe sur la capacité de réaction des analystes.

II.2.g L’exécution asynchrone des tâches : Celery

Notre besoin : Nous devons éviter que certains traitements, notamment les analyses lourdes, ne bloquent l’interface ni ne ralentissent les autres fonctions du système.

Explication simple : Celery permet d’exécuter des tâches en arrière-plan. Le serveur envoie le traitement à effectuer à un service dédié, puis poursuit ses autres activités sans attendre la fin du calcul.

Solution comparée	Avantages	Limites	Statut
Exécution directe dans la requête	Développement plus simple.	Peut bloquer l’interface et dégrader l’expérience utilisateur.	Écarté
Traitement asynchrone avec Celery	Préserve la fluidité du service pendant les calculs longs.	Ajoute un composant supplémentaire à administrer.	Retenu

TABLE II.12 – Comparaison des modes d’exécution des traitements longs

Justification : L’usage de Celery améliore la disponibilité perçue du système et évite qu’un traitement coûteux ne bloque des actions simples.

II.3 Les bases de données et les services de stockage

II.3.a La base relationnelle principale : PostgreSQL

Notre besoin : Nous devons stocker les données sensibles de la plateforme dans un système fiable, structuré et apte à garantir leur intégrité.

Explication simple : PostgreSQL est une base de données relationnelle. Elle organise les informations dans des tables définies à l'avance et applique des règles strictes sur leur cohérence.

Solution comparée	Avantages	Limites	Statut
Stockage principal en NoSQL	Souplesse de modélisation.	Moins adapté, dans ce cas, aux contraintes fortes d'intégrité et de structure.	Écarté
PostgreSQL	Très bon niveau de fiabilité, transactions robustes et structure claire des données.	Schéma à définir avec précision en amont.	Retenu

TABLE II.13 – Comparaison des solutions de stockage structuré

Justification : PostgreSQL convient aux données critiques de la plateforme, notamment celles qui exigent cohérence, traçabilité et règles de validation strictes.

II.3.b Le stockage documentaire : MongoDB

Notre besoin : Les contenus que nous collections présentent des structures variables. Nous avons donc besoin d'une solution capable de stocker des documents hétérogènes sans imposer un schéma rigide.

Explication simple : MongoDB stocke les informations sous forme de documents. Ce mode de stockage s'adapte bien aux contenus dont les champs peuvent varier selon la source.

Solution comparée	Avantages	Limites	Statut
Stockage intégral dans PostgreSQL	Centralisation dans une seule technologie.	Moins souple face à des documents de structure très variable.	Écarté
MongoDB	Bonne adaptation aux contenus hétérogènes et évolution de schéma plus simple.	Moins adapté aux besoins fortement relationnels.	Retenu

TABLE II.14 – Comparaison des solutions de stockage documentaire

Justification : MongoDB répond au besoin de souplesse imposé par les données issues de la collecte, dont la forme dépend fortement des sources observées.

II.3.c La base orientée graphe : Neo4j

Notre besoin : Nous devons représenter et analyser rapidement les relations entre acteurs, infrastructures, événements et indicateurs.

Explication simple : Neo4j est une base de données orientée graphe. Elle enregistre des nœuds et des relations, ce qui facilite l'exploration de liens complexes entre plusieurs entités.

Solution comparée	Avantages	Limites	Statut
Base relationnelle classique	Technologie déjà maîtrisée dans de nombreux contextes.	Les requêtes sur des chaînes de relations complexes deviennent rapidement coûteuses.	Écarté
Neo4j	Requêtes efficaces sur les relations et représentation naturelle des réseaux d'entités.	Introduit un modèle de données et un langage de requête spécifiques.	Retenu

TABLE II.15 – Comparaison des solutions d'analyse relationnelle

Justification : Neo4j est adapté à l'analyse de liens, qui constitue une fonction centrale dans une plateforme de renseignement sur les menaces.

II.3.d Le moteur de recherche textuelle : Elasticsearch

Notre besoin : Nous devons permettre aux utilisateurs de retrouver rapidement un terme, un indicateur ou un document parmi un grand volume de contenus.

Explication simple : Elasticsearch crée des index de recherche spécialisés. Ces index accélèrent fortement la recherche par mots, expressions ou filtres.

Solution comparée	Avantages	Limites	Statut
Recherche simple en base	Déploiement plus simple.	Performances insuffisantes sur de grands corpus textuels.	Écarté
Elasticsearch	Recherche rapide, filtres avancés et bonne adaptation aux volumes élevés.	Nécessite des index supplémentaires et une administration dédiée.	Retenu

TABLE II.16 – Comparaison des solutions de recherche textuelle

Justification : Un moteur de recherche dédié est nécessaire pour garantir un accès rapide à l'information au cours des investigations.

II.3.e La base vectorielle : Qdrant

Notre besoin : Nous devons retrouver des contenus proches par le sens et non uniquement par correspondance exacte de mots.

Explication simple : Qdrant stocke des représentations numériques appelées vecteurs. Deux textes de sens proche produisent des vecteurs proches, ce qui permet une recherche sémantique.

Solution comparée	Avantages	Limites	Statut
Recherche lexicale classique	Bonne précision sur des termes exacts.	Capacité limitée à rapprocher des textes formulés différemment mais de sens voisin.	Écarté
Qdrant	Prend en charge la recherche sémantique utile aux fonctions d'assistance et de recommandation.	Technologie plus récente et plus spécialisée.	Retenu

TABLE II.17 – Comparaison des solutions de recherche sémantique

Justification : L'usage d'une base vectorielle complète utilement la recherche textuelle classique, en particulier pour les interactions en langage naturel.

II.3.f Le cache mémoire : Redis

Notre besoin : Nous devons rendre certaines données fréquemment consultées très rapidement accessibles afin de limiter les lectures répétées sur les bases principales.

Explication simple : **Redis** stocke temporairement des informations en mémoire. Ce fonctionnement réduit le temps d'accès à des données réutilisées souvent.

Solution comparée	Avantages	Limites	Statut
Lecture directe sur les bases principales	Architecture plus simple.	Risque de surcharge et temps de réponse plus élevés pour des demandes répétées.	Écarté
Redis	Accès rapide aux données temporaires et réduction de la charge sur les autres services.	Les données en cache sont temporaires et doivent être régénérées si nécessaire.	Retenu

TABLE II.18 – Comparaison des mécanismes de cache mémoire

Justification : Redis améliore la réactivité de la plateforme sur les consultations fréquentes tout en protégeant les services de stockage principaux contre des sollicitations répétitives.

II.4 L'interface utilisateur et les visualisations

II.4.a Le cadre de développement de l'interface : *Next.js* et *React*

Notre besoin : Nous voulions que le tableau de bord s'affiche rapidement, reste fluide et offre une expérience homogène sur différents postes de travail.

Explication simple : **React** sert à construire les composants de l'interface. **Next.js** organise leur rendu et améliore le chargement initial des pages.

Solution comparée	Avantages	Limites	Statut
React seul	Mise en place directe pour une interface monopage.	Moins favorable, ici, au chargement optimisé et à la structuration globale du rendu.	Écarté
Next.js avec React	Meilleure organisation du rendu, chargement initial optimisé et architecture front-end plus complète.	Configuration plus importante côté application.	Retenu

TABLE II.19 – Comparaison des cadres de développement de l'interface web

Justification : Next.js et React forment un ensemble cohérent pour produire une interface moderne, structurée et suffisamment réactive pour un usage intensif.

II.4.b Le style, l'accessibilité et l'état local : Tailwind CSS, Radix UI et Zustand

Notre besoin : Nous voulions que l'interface reste lisible, accessible au clavier, cohérente visuellement et capable de conserver certains états d'usage côté client.

Explication simple : **Tailwind CSS** facilite la mise en forme de l'interface. **Radix UI** apporte des composants pensés pour l'accessibilité. **Zustand** gère des états locaux simples dans l'application.

Solution comparée	Avantages	Limites	Statut
Thèmes génériques prêts à l'emploi	Déploiement visuel rapide.	Personnalisation limitée et contrôle réduit sur l'accessibilité fine.	Écarté
Tailwind CSS + Radix UI + Zustand	Bon équilibre entre personnalisation, accessibilité et gestion légère de l'état local.	Demande un travail de composition plus détaillé.	Retenu

TABLE II.20 – Comparaison des solutions de construction de l'interface

Justification : Cet ensemble permet de produire une interface claire, accessible et adaptée aux besoins spécifiques du projet sans alourdir inutilement l'architecture.

II.4.c Les animations d'interface : Framer Motion

Notre besoin : Nous voulions intégrer à l'interface des transitions visuelles claires afin d'améliorer la lisibilité des changements d'état et le confort d'utilisation.

Explication simple : **Framer Motion** est une bibliothèque d'animation pour interfaces web. Elle permet d'ajouter des transitions fluides entre les éléments affichés.

Solution comparée	Avantages	Limites	Statut
Animations développées manuellement	Contrôle complet sur chaque effet.	Coût de développement plus élevé et maintenance plus délicate.	Écarté
Framer Motion	Mise en œuvre rapide d'animations cohérentes et maintenables.	Ajoute une dépendance supplémentaire au projet.	Retenu

TABLE II.21 – Comparaison des solutions d'animation de l'interface

Justification : Des transitions bien maîtrisées améliorent la compréhension de l’interface et réduisent les ruptures visuelles lors des interactions.

II.4.d La visualisation relationnelle en trois dimensions : Cytoscape.js, Three.js et Force-Graph

Notre besoin : Nous devons représenter certaines données relationnelles complexes de manière exploitable, même lorsque le nombre de liens devient élevé.

Explication simple : **Cytoscape.js** aide à structurer les graphes. **Three.js** et **Force-Graph** servent à afficher des représentations interactives en trois dimensions dans le navigateur.

Solution comparée	Avantages	Limites	Statut
Visualisation 2D classique	Mise en œuvre plus simple et coût de rendu plus faible.	Lisibilité réduite lorsque la densité des relations augmente fortement.	Écarté
Visualisation 3D	Offre plus d’espace de représentation et facilite l’exploration interactive de graphes denses.	Réalisation plus complexe et demande graphique plus élevée.	Retenu

TABLE II.22 – Comparaison des solutions de visualisation relationnelle

Justification : Pour les ensembles relationnels complexes, une représentation 3D améliore la lecture visuelle et l’exploration des liens entre entités.

II.4.e La cartographie géospatiale : Mapbox, Deck.gl et GeoIP

Notre besoin : Nous devons localiser et représenter géographiquement certaines activités observées afin d’en faciliter l’interprétation.

Explication simple : **GeoIP** associe une adresse IP à une localisation estimée. **Mapbox** fournit le fond cartographique. **Deck.gl** permet d’ajouter des couches de visualisation avancées sur cette carte.

Solution comparée	Avantages	Limites	Statut
Cartographie web simple	Intégration rapide.	Capacités plus limitées pour des couches visuelles avancées et interactives.	Écarté
Mapbox avec Deck.gl	Bon niveau de personnalisation et visualisations géospatiales plus riches.	Paramétrage plus complexe.	Retenu

TABLE II.23 – Comparaison des solutions de cartographie géospatiale

Justification : Ce choix permet de représenter les données géographiques avec un niveau de détail suffisant pour l'analyse tout en conservant une interface exploitable.

II.4.f Les graphiques statistiques : *Recharts* et *Chart.js*

Notre besoin : Nous voulions fournir aux utilisateurs des graphiques simples pour suivre l'évolution des événements, des volumes et des indicateurs principaux.

Explication simple : *Recharts* et *Chart.js* permettent de produire rapidement des graphiques adaptés à une interface web, à partir de séries de données structurées.

Solution comparée	Avantages	Limites	Statut
Développement graphique manuel	Liberté totale de conception.	Temps de développement plus important et maintenance plus difficile.	Écarté
<i>Recharts</i> et <i>Chart.js</i>	Production rapide de graphiques lisibles et adaptés au web.	Personnalisation parfois encadrée par les composants disponibles.	Retenu

TABLE II.24 – Comparaison des solutions de visualisation statistique

Justification : Ces bibliothèques répondent efficacement au besoin de synthèse visuelle, avec un bon compromis entre rapidité de développement et qualité d'affichage.

II.4.g La génération de rapports PDF : *Puppeteer* et *Express*

Notre besoin : Nous voulions que les analystes puissent produire un document PDF reprenant fidèlement les contenus affichés dans l'interface.

Explication simple : *Express* prépare le service nécessaire à la génération du document. *Puppeteer* ouvre une page web de manière automatisée et en produit une version PDF proche du rendu observé à l'écran.

Solution comparée	Avantages	Limites	Statut
Génération PDF manuelle	Architecture plus simple sur certains cas.	Reproduction incomplète des graphiques et des composants complexes.	Écarté
Puppeteer avec Express	Restitution fidèle du rendu visuel de l'interface dans le document final.	Temps de génération un peu plus élevé.	Retenu

TABLE II.25 – Comparaison des méthodes de production des rapports PDF

Justification : La fidélité du rendu est essentielle pour produire des rapports lisibles et directement exploitables par des décideurs ou des partenaires externes.

II.5 La sécurité, la documentation et l'exploitation

II.5.a Le contrôle d'accès et la validation des entrées : JWT, RBAC et Zod

Notre besoin : Nous devons contrôler les droits d'accès et vérifier les données reçues afin de réduire les risques d'usage non autorisé ou de saisie malveillante.

Explication simple : **JWT** gère l'authentification par jetons. **RBAC** définit les droits selon le rôle de chaque utilisateur. **Zod** valide la structure des données avant leur traitement.

Solution comparée	Avantages	Limites	Statut
Contrôles d'accès minimaux	Déploiement plus simple.	Niveau de sécurité insuffisant pour des données sensibles.	Écarté
JWT + RBAC + Zod	Contrôle plus strict des accès et meilleure validation des entrées.	Exige une implémentation rigoureuse sur l'ensemble de l'application.	Retenu

TABLE II.26 – Comparaison des mécanismes de sécurité applicative

Justification : Le niveau de sensibilité des données traitées impose des contrôles d'accès explicites et une validation systématique des entrées utilisateur.

II.5.b La documentation interactive : OpenAPI et Swagger

Notre besoin : Nous devons fournir aux partenaires techniques une documentation claire, à jour et exploitable pour intégrer leurs propres outils à la plateforme.

Explication simple : **OpenAPI** décrit formellement les interfaces du service. **Swagger** génère ensuite une documentation interactive à partir de cette description.

Solution comparée	Avantages	Limites	Statut
Documentation rédigée séparément	Facile à produire au départ.	Risque élevé d'écart entre le document et le service réel.	Écarté
OpenAPI et Swagger	Documentation synchronisée avec l'API et plus simple à exploiter pour les intégrateurs.	Suppose une annotation rigoureuse des interfaces.	Retenu

TABLE II.27 – Comparaison des méthodes de documentation de l'API

Justification : Une documentation générée à partir de la définition technique du service réduit les incohérences et facilite l'adoption par les partenaires.

II.5.c Le démarrage et l'orchestration des services : scripts Batch et Docker Compose

Notre besoin : Nous devons pouvoir lancer l'ensemble applicatif de manière simple, répétable et ordonnée, sans dépendre d'une procédure manuelle complexe.

Explication simple : Les scripts de lancement déclenchent automatiquement les commandes nécessaires. **Docker Compose** organise le démarrage coordonné des différents services utilisés par la plateforme.

Solution comparée	Avantages	Limites	Statut
Démarrage manuel des services	Aucun outil d'orchestration supplémentaire.	Procédure plus longue, plus fragile et source d'erreurs.	Écarté
Scripts Batch et Docker Compose	Lancement reproductible, ordre d'exécution maîtrisé et exploitation simplifiée.	Préparation initiale plus structurée.	Retenu

TABLE II.28 – Comparaison des méthodes de démarrage de la plateforme

Justification : Ce choix améliore l'exploitabilité de la solution, limite les erreurs de démarrage et facilite les démonstrations comme les déploiements internes.